Universidad Rafael Landívar
Ciencia de Datos
Segundo Semestre 2026
Ing. Max Cerna

Agencia Metropolitana de Transporte
¿Quién deja de usar la red?

Modalidad	Ponderación	Fases	Defensa
Grupos de 3 o 4 integrantes	100 puntos	Fase 1: datos y modelo base Fase 2: modelos, MLOps y defensa	Oral, obligatoria 9 de noviembre

Contexto

En el Proyecto 1 construimos la plataforma de datos de la Agencia, donde integramos a los cuatro operadores y el padrón con sus controles de calidad, ahora la dirección quiere usarla para algo concreto.

Cada mes una parte de los usuarios deja de usar la red: algunos cancelan su tarjeta, pero la mayoría simplemente deja de viajar sin avisar.

La Agencia tiene presupuesto para una campaña de retención (por ejemplo, un mensaje con una recarga de cortesía), pero no alcanza para todos, así que hay que decidir a quién contactar.

> **La pregunta de este proyecto**
> Con lo que sabemos de un usuario hasta hoy, ¿qué tan probable es que deje de usar la red en los próximos 30 días?
>
> Y con esa probabilidad, ¿a quién conviene contactar?

Los datos son simulados, igual que en el Proyecto 1

El problema

Es un problema de clasificación binaria: para cada usuario y fecha de corte (el día en que la Agencia decide a quién contactar), predices si dejará de usar la red en los 30 días siguientes.

Todo lo que el modelo ve debe ser anterior a la fecha de corte, y la etiqueta (la respuesta que debe predecir: si el usuario dejó de viajar o no) se calcula solo con datos desde esa fecha en adelante.

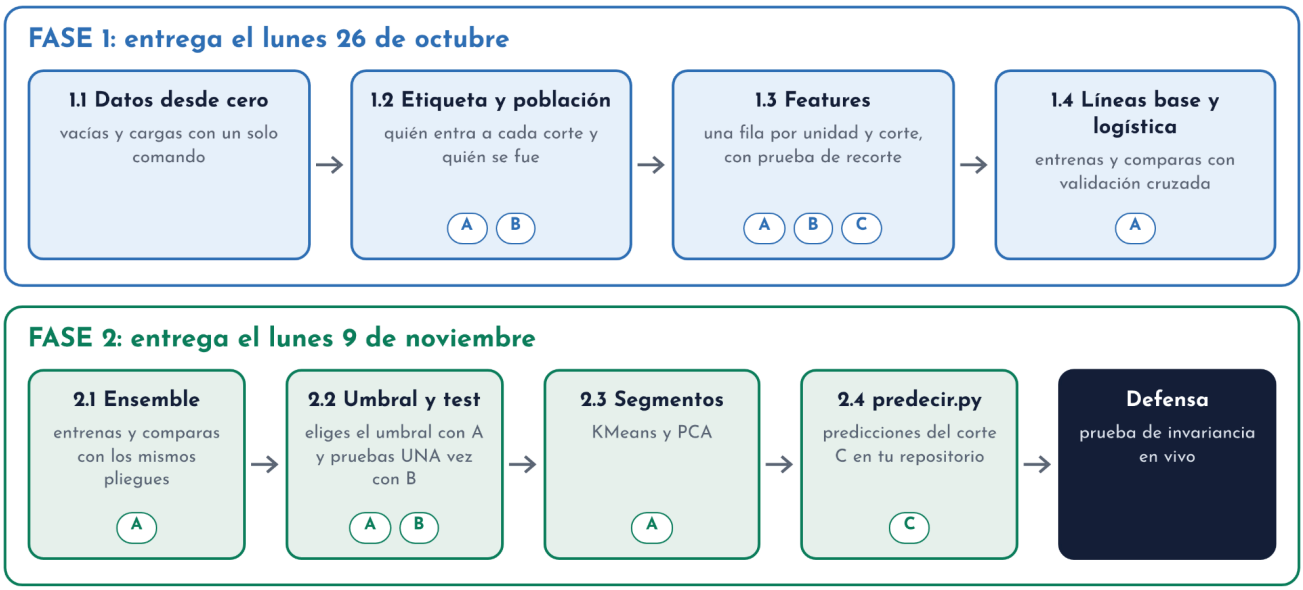
Usar información que no existía en la fecha de corte se considera una fuga de datos.

Flujo esperado

El trabajo nuevo se construye sobre tu capa Silver del Proyecto 1:

Paso	Qué hace
Datos nuevos	Vacías tu lake, tu warehouse y tus snapshots, y corres tu pipeline del Proyecto 1 con los datos nuevos, sin reestructurarlo
Etiqueta	1 si la unidad no tuvo ningún abordaje válido en los 30 días que empiezan en el corte y 0 si viajó (dejar de viajar no es lo mismo que darse de baja en el CDC)
Features	Las variables con las que el modelo predice: una fila por unidad y fecha de corte, calculada solo con datos anteriores al corte
Modelos	Una línea base (baseline: un modelo muy simple que sirve de punto de comparación), una regresión logística regularizada y un ensemble (un modelo que combina muchos árboles, como Random Forest o boosting), comparados con validación cruzada
Decisión	Eliges el umbral (la probabilidad a partir de la cual contactas a alguien) con el escenario de costos y evalúas el modelo una sola vez en el test
Segmentos	KMeans y PCA sobre la misma tabla de features
Producción	El script predecir.py, el modelo guardado y la ficha del modelo (un documento que describe el modelo, sus datos y sus límites)

Ruta del proyecto



Las letras A, B y C de cada tarjeta indican qué cortes usa ese paso.

Regla para recordar: entrenas y ajustas con A, te evalúas una vez con B y entregas C

Datos del Proyecto 2

Recibirás los datos en un .zip con los mismos nueve archivos del Proyecto 1: mismos nombres, columnas, formatos y problemas de calidad.

Se mantiene igual	Cambia
Los 9 archivos, sus columnas y formatos	El periodo: 90 días, del 1 de junio al 29 de agosto de 2026
Las llaves de cada operador	Hay usuarios nuevos, con números de tarjeta que no existían
Los catálogos de estaciones y paradas	Los usuarios tienen hábitos propios y algunos dejan de usar la red
Duplicados, nulos, fechas futuras, viajes sin salida	Los DELETE del CDC corresponden a bajas reales
El CDC con INSERT, UPDATE y DELETE	El tamaño: unos 270 MB

Parte de tu trabajo es descubrir cómo se ven en los datos los usuarios que se van y qué los distingue.

Cómo cargarlos: los datos nuevos **reemplazan** a los del Proyecto 1, no se agregan, si los cargas encima, tu Bronze, tus snapshots o tus tablas incrementales mezclarán las dos versiones sin avisarte.

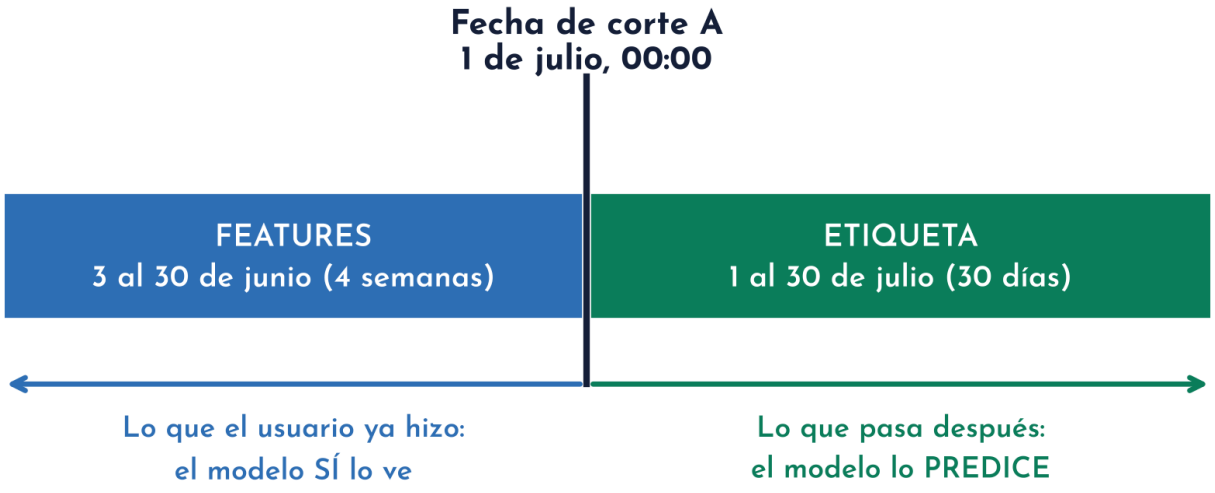
- Vacía tu lake, tu warehouse, tus snapshots de dbt y cualquier tabla incremental.
- Pon los archivos nuevos en datos_red/.
- Corre tu flujo completo con un solo comando; si la primera vez necesitó pasos manuales (cargar seeds, crear tablas), agrégalos al flujo.
- Compara tus conteos de Bronze con la tabla de conteos esperados que se publica junto con los datos.

Fechas de corte y etiqueta

¿Qué es una fecha de corte?

Es el día en que la Agencia decide a quién contactar, cuando solo conoce el pasado. Piensa en cada corte como una foto tomada ese día:

- **Mirando hacia atrás (4 semanas):** lo que el usuario hizo, de donde salen las features.
- **Mirando hacia adelante (30 días):** si el usuario siguió viajando, de donde sale la etiqueta (1 si no viajó, 0 si viajó).

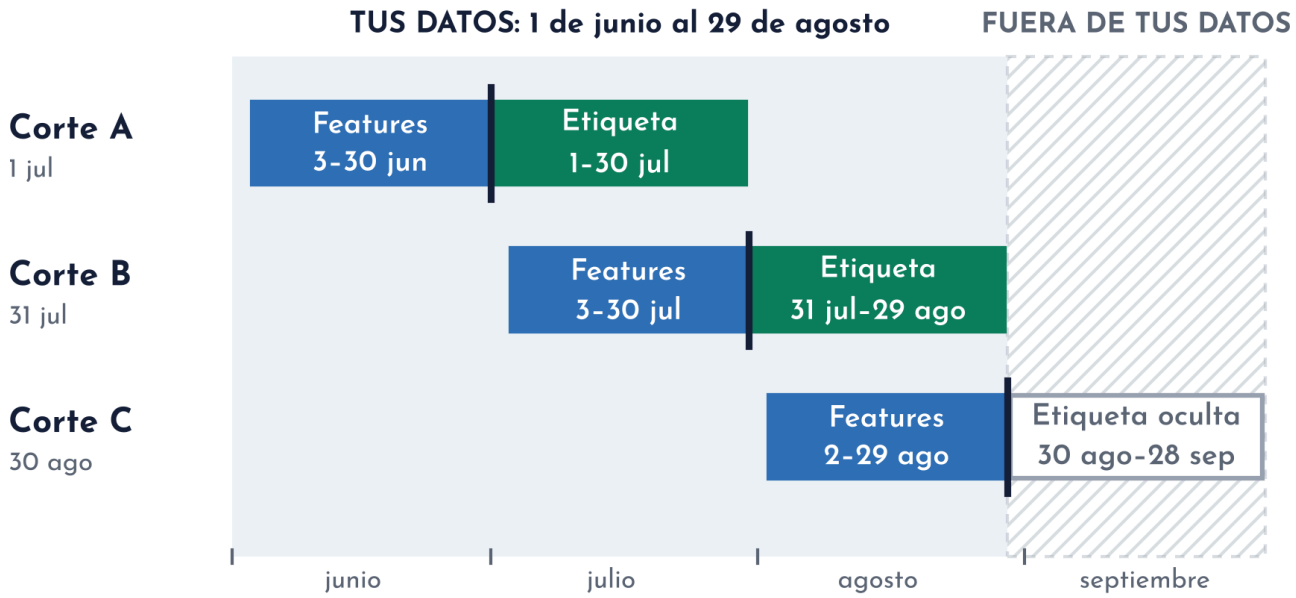


La regla exacta:

La fecha de corte t son las 00:00 (hora local) del día de la decisión: las features usan solo eventos anteriores a t y la etiqueta usa los 30 días que empiezan en t.

Los tres cortes del proyecto

Tus datos van del 1 de junio al 29 de agosto, y cada corte es la misma foto tomada en otra fecha:



¿Para qué sirve cada corte si solo entrenas con el A?

Los tres cortes se calculan con el mismo código y solo cambia la fecha, pero cada uno cumple un papel distinto:

Corte	Papel	En palabras simples
A	Entrenamiento	Es donde el modelo aprende: ves junio y ya sabes qué pasó en julio, así que tienes preguntas y respuestas.
B	Test	Es tu examen final: con filas que el modelo nunca vio, te dice si lo que aprendió sirve un mes después. Si lo usas para entrenar o ajustar, deja de servir para eso.

C	Uso real	Es como usar el modelo de verdad: tienes agosto, pero septiembre todavía no pasó. Entregas tus predicciones y el catedrático las compara con lo que ocurrió.

La ventana de una feature es el periodo con el que se calcula, en los tres cortes mide 28 días (4 semanas completas), la más larga que cabe en el corte A, si un corte usará ventanas más largas que otro, la misma feature mediría cosas distintas al entrenar y al evaluar.

Población: entran a un corte las unidades con al menos 2 abordajes válidos en los 28 días anteriores y sin un DELETE del CDC aplicado antes del corte.

Etiqueta: positivo (1) si la unidad no tiene ningún abordaje válido, en ningún operador, en los 30 días que empiezan en el corte, y negativo (0) en otro caso.

Unidad: es lo que representa cada fila de tu tabla, y tú decides si es la persona (tarjetas enlazadas con tu estrategia de identidad del Proyecto 1) o la tarjeta.

Las dos opciones son válidas, pero cada una tiene un costo que debes medir: por ejemplo, con la tarjeta, un usuario que cambia de operador sin dejar la red aparece como abandono.

FASE 1: Datos, etiqueta y modelo base

1.1 Datos nuevos desde cero (5 pts)

Carga los datos del Proyecto 2 con tu pipeline del Proyecto 1, desde un entorno vacío y con un solo comando, no se trata de rehacer tu arquitectura, sino de demostrar que corre desde cero y carga lo que debe.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Comando único	Una línea en el README que corre todo desde un entorno vacío	Lo corremos y termina sin pasos manuales
Tabla de conteos	9 filas, una por archivo: archivo, filas esperadas, filas cargadas y diferencia	Toda diferencia distinta de 0 tiene su explicación
Tiempos	Una fila por etapa: etapa y segundos	Están todas las etapas del flujo

1.2 Etiqueta, población y unidad (8 pts)

- Declara tu unidad y justifícala con lo que hiciste en el Proyecto 1.
- Construye la población y la etiqueta de los cortes A y B a partir de Silver, igual que las features.
- Reporta por corte la población, los positivos y la tasa positiva (positivos / población) con la cuenta completa, por ejemplo: 4,200 / 50,000 = 0.084.
- Mide el costo de tu unidad: si es la tarjeta, cuántos positivos son de personas que siguieron viajando con otra tarjeta (mismo número en otro operador); si es la persona, cómo trataste Aerómetro y qué tarjetas no pudiste enlazar.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Definición de la etiqueta	Una sola frase	Se puede calcular sólo con esa frase
Unidad elegida	Persona o tarjeta, y 3 a 5 líneas de justificación	Está conectada con lo que hiciste en el Proyecto 1
Tabla de población	2 filas (A y B): corte, población, positivos, tasa positiva y la cuenta	La cuenta está escrita completa, por ejemplo: 4,200 / 50,000 = 0.084
Costo de la unidad	Un número y su proporción, por ejemplo: 420 de 4,200 positivos = 0.10	Sale de tus datos y explicas cómo lo contaste

1.3 Features a la fecha de corte y auditoría de fuga (12 pts)

Ajusta la tabla de features por usuario del Proyecto 1 (sección 2.3) para poder entrenar un modelo:

- Tu código recibe la fecha de corte como parámetro.
- Una fila por unidad en cada corte (quien está en los tres cortes tiene tres filas), todo en una misma tabla con la fecha de corte como columna.
- Todas las ventanas miden lo mismo en los tres cortes (máximo 28 días).
- Usa solo registros válidos: lo que está en cuarentena no entra.

Ejemplo (valores inventados): en la fila de P123 del corte A, las features se calculan con sus viajes del 3 al 30 de junio y la etiqueta con sus viajes del 1 al 30 de julio.

unidad_id	fecha_corte	viajes_28d	dias_desde_ultimo_viaje	etiqueta
P123	2026-07-01	38	1	0
P123	2026-07-31	35	2	0
P456	2026-07-01	12	2	0

Cada corte usa su etiqueta de forma distinta: en el A la calculas y entrenas con ella, en el B la calculas pero no la usas hasta la sección 2.2, y en el C no puedes calcularla porque esos días no están en tus datos.

Auditoría de fuga: Calcula las features del corte A dos veces, con tus datos completos y con tus datos recortados al 30 de junio. Esta prueba de recorte debe dar dos tablas idénticas; si no, alguna feature usa datos posteriores al corte.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Tabla de features	Columnas unidad_id, fecha_corte, tus features y etiqueta (vacía en C). Filas de los cortes A, B y C	Están los tres cortes y una fila por unidad en cada corte
Diccionario	Una fila por feature: nombre, qué mide, ventana y tabla de Silver de donde sale	Están todas las features y ninguna ventana pasa de 28 días
Prueba de recorte	Filas comparadas, número de diferencias (debe ser 0) y la lista de features que fallaron antes de corregirlas	Se puede repetir con tu código

1.4 Baseline y regresión logística regularizada (15 pts)

- Al menos dos baseline, el primero le da a todos la misma probabilidad, igual a la tasa positiva del corte A que calculaste en 1.2, es decir, la proporción de usuarios que dejaron de viajar.

Si en la población de A hay 50,000 usuarios y 4,200 tienen etiqueta 1, la tasa es 4,200 / 50,000 = 0.084, y todos reciben 0.084, como si le dijeras a cada uno "no sé nada de ti, así que te doy el promedio".

El segundo usa una sola feature como puntaje, sin entrenar nada: eliges una que creas relacionada con el abandono, justificas por qué y ordenas a los usuarios por ella (por ejemplo, quien hizo menos viajes en las 4 semanas antes del corte queda con más riesgo).

Evalúa los dos con los mismos folds y la misma métrica que tus modelos.

- Entrena una regresión logística regularizada: compara al menos dos tipos de castigo (L2, L1 o Elastic Net), elige λ con validación cruzada y gráfica tu métrica de validación contra λ.
- Usa validación cruzada estratificada de 5 folds, **solo con el corte A**, elige y justifica la métrica con la que comparas modelos, y reporta su media y desviación estándar.
- Interpreta los coeficientes: cuáles empujan hacia el abandono y si eso tiene sentido.
- No evalúes nada con la etiqueta del corte B en esta fase.

Notación: recuerda que λ es el castigo de la regularización y en LogisticRegression de scikit-learn el castigo se controla con C = 1 / λ: un C grande castiga poco.

En Ridge, Lasso y ElasticNet el parámetro alpha sí es λ.

α queda para la tasa de aprendizaje del descenso de gradiente.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Métrica elegida	3 a 5 líneas: qué métrica y por qué	La justificación se relaciona con el problema
Tabla de modelos	Al menos 4 filas (2 líneas base y 2 logísticas): modelo, métrica media y desviación estándar	Todos usan los mismos 5 folds del corte A
Gráfica	Tu métrica contra λ	Se ve qué λ elegiste
Coeficientes	Los 5 que más empujan hacia el abandono, con una línea de interpretación cada uno	La interpretación coincide con el signo

FASE 2: Modelos, decisión y producción

2.1 Ensemble y sesgo-varianza (10 pts)

- Entrena al menos un ensemble (bagging como Random Forest, o boosting) con los mismos folds de la Fase 1, para que la comparación sea justa.
- Compara el error de entrenamiento con el de validación y explica en qué lado del balance sesgo-varianza está cada modelo.
- Calcula la importancia de variables por permutación en validación, no en entrenamiento.
- Decide: ¿la mejora del ensemble sobre la logística justifica su complejidad?

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Tabla de modelos	La tabla de 1.4 con una fila más por ensemble, y las columnas métrica en entrenamiento y métrica en validación	Mismos folds que en 1.4
Sesgo-varianza	Una línea por modelo: ¿sesgo alto, varianza alta o equilibrado? ¿Por qué?	Coincide con la diferencia entre entrenamiento y validación
Importancias	Las 10 features más importantes por permutación	Se calcularon en validación
Decisión	3 a 5 líneas: qué modelo va a producción y por qué	Compara la mejora contra la complejidad

2.2 Umbral según costo y test único (12 pts)

Escenario de la Agencia (valores simulados, los mismos para todos los grupos): contactar a un usuario cuesta Q5, si el usuario se iba a ir, la campaña lo retiene en el 25% de los casos, y un usuario retenido vale Q60.

Con este escenario, escribe la ganancia de contactar a un grupo de usuarios y deriva de ahí tu umbral, con la cuenta completa.

Elige el umbral con las predicciones de validación cruzada del corte A, después, y **una sola vez**, entrena el modelo final con todo el corte A y evalúalo en el corte B. Anota en tu bitácora la fecha y el commit de esa evaluación, y si el resultado es peor que en validación, explícalo sin volver a ajustar.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Ganancia	La fórmula de ganancia que derivaste, con la cuenta completa	Usa los valores del escenario
Curva	Ganancia contra umbral, con el umbral elegido marcado	El umbral elegido es el que muestra la curva
Test B	Fecha y commit de la única evaluación, tu métrica, matriz de confusión y ganancia en quetzales	Hay una sola evaluación registrada

2.3 Segmentación con KMeans y PCA (8 pts)

- Aplica KMeans a la tabla de features del corte A, ya escalada. Elige k con el método del codo y la silueta, y justifica tu elección.
- Visualiza los segmentos con PCA a dos componentes y reporta la varianza explicada.
- Describe cada segmento con medianas de sus features y ponle un nombre.
- Muestra la tasa de abandono por segmento, que es descriptiva: no prueba que el segmento cause el abandono.
- Si usas el segmento como feature del clasificador, ajusta KMeans dentro de la validación cruzada.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
Elección de k	Gráfica del codo, gráfica de la silueta y el k elegido	El k coincide con las gráficas o explicas por qué no
Gráfico PCA	Los segmentos en dos componentes, con la varianza explicada	Los datos están escalados
Tabla de segmentos	segmento, nombre, tamaño, medianas de las features principales y tasa de abandono	Cada segmento tiene nombre
Acciones	Una acción de retención para al menos dos segmentos	Son distintas entre sí

2.4 MLOps: predecir, guardar y documentar (12 pts)

- predecir.py recibe la carpeta de datos y la fecha de corte, y produce un CSV con las columnas unidad_id, tarjetas (las llaves originales de la unidad, separadas por ;), puntaje y contactar (0 o 1), que entregas en tu repositorio de GitHub.
- Guarda el modelo con su versión y cárgalo en predecir.py, que no debe reentrenarlo.
- Entorno reproducible: versiones fijas de las librerías, semilla fija y un README que permita correrlo en otra máquina.
- Ficha del modelo: datos y periodo, unidad, población, etiqueta, métricas en validación y en B, umbral y su costo, fugas revisadas, limitaciones y el aviso de que los datos son simulados.
- Entrega tus predicciones para el corte C (30 de agosto): ya tienes los datos para calcularlas y solo la etiqueta está oculta.

Prueba de invariancia: El día de la calificación recibirás los mismos datos con 30 días más y correrás tu flujo y predecir.py con corte 30 de agosto, tus predicciones deben ser idénticas a las que entregaste, si cambian, tu pipeline usa datos posteriores al corte.

Qué entregas y cómo se revisa:

Entregable	Qué debe contener	Cómo se revisa
predecir.py	Corre con: python predecir.py --datos <carpeta> --fecha-corte 2026-08-30	Lo corremos y genera el CSV
CSV de predicciones del corte C	Columnas unidad_id, tarjetas, puntaje y contactar. Una fila por unidad de la población de C	Está en tu repositorio de GitHub
Modelo guardado	Un archivo con la versión en el nombre	predecir.py lo carga y no reentrena
Ficha del modelo	Una página con: datos y periodo, unidad, población, etiqueta, métricas en validación y en B, umbral y su costo, fugas revisadas, limitaciones y el aviso de datos simulados	Están todos los campos

Presentación y defensa

Momento	Qué pasa	Duración
Presentación	El problema, tus decisiones y tus resultados	15 minutos
Prueba de invariancia	Corres tu flujo y predecir.py con 30 días más de datos y comparas tus predicciones con las que entregaste	En vivo
Decisión ajena	Cada integrante explica una decisión que tomó otra persona del equipo: qué se decidió, qué alternativas había y por qué se descartaron	Cada integrante
Preguntas de fondo	Dónde pudo haber fuga en tu proyecto y cómo lo sabes; por qué esa métrica y de dónde salió tu umbral	Al final

Tecnologías

La opción recomendada es la que tendrá soporte en el curso, puedes usar otra, pero debes justificar el cambio en tu documentación.

Pieza	Recomendada	Alternativas	Nota
Modelos	scikit-learn	statsmodels	Pipeline y ColumnTransformer evitan la fuga en la validación
Boosting	HistGradientBoosting Classifier	XGBoost, LightGBM	learning_rate cumple el papel de α
Guardado	joblib	pickle, ONNX	Guarda el pipeline completo, no solo el modelo
Seguimiento	bitácora en el repositorio	MLflow	Toda corrida que reportes debe poder repetirse
Orquestación	la misma del Proyecto 1	Prefect, Airflow, Dagster	Agrega las tareas nuevas a tu flujo

Métricas que debes reportar

Igual que en el Proyecto 1, lo que no está medido no cuenta:

Categoría	Qué medir
Problema	Población, positivos y tasa positiva por corte, con la cuenta completa
Validación	La métrica que elegiste y por qué, con su media y desviación estándar en validación cruzada, siempre junto a la línea base
Decisión	Umbral, precisión, recall y ganancia en quetzales con el escenario de costos
Test	Resultados del corte B, con la fecha y el commit de la única evaluación
Fuga	Resultado de la prueba de recorte y de la prueba de invariancia
Rendimiento	Tiempo de carga desde cero, de cálculo de features, de entrenamiento y de predicción

Un modelo sin su baseline al lado no dice nada

Evaluación

Componente	Puntos
1.1 Datos nuevos desde cero	5
1.2 Etiqueta, población y unidad	8
1.3 Features a la fecha de corte y auditoría de fuga	12
1.4 Baseline y logística regularizada	15
2.1 Ensemble y sesgo-varianza	10
2.2 Umbral según costo y test único	12
2.3 Segmentación con KMeans y PCA	8
2.4 MLOps: predecir, guardar y documentar	12
Presentación y defensa	18
Total	**100**

Rúbrica

Componente	Completo	Parcial	Sin puntos
1.1	Corre desde cero con un comando y los conteos cuadran o se explican	Corre con pasos manuales o hay diferencias sin explicar	No carga los datos nuevos o los mezcla con los viejos
1.2	Etiqueta y población exactas, tasa por corte y costo de la unidad medido	Etiqueta correcta, pero sin medir el costo de la unidad	Etiqueta que usa datos anteriores al corte o población sin definir
1.3	Corte como parámetro, tres cortes, prueba de recorte idéntica y fugas documentadas	Tabla correcta sin la prueba de recorte, o ventanas que no caben en el corte A	Fuga confirmada en las features
1.4	Dos líneas base, métrica justificada, castigos comparados, λ elegido por validación y coeficientes interpretados	Logística sin comparar castigos o sin elegir λ	Sin línea base o evaluado con el corte B
2.1	Comparación con los mismos folds, sesgo-varianza explicado y decisión justificada	Ensemble entrenado sin análisis de sesgo-varianza o sin decisión	Sin ensemble o con otros folds
2.2	Umbral derivado del costo con la cuenta completa y test B usado una vez	Umbral por costo sin mostrar de dónde sale	Umbral sin justificar o test usado más de una vez
2.3	k justificado, segmentos descritos y acciones por segmento	Segmentos sin descripción o k sin justificar	KMeans ajustado con datos de test o sin escalar
2.4	predecir.py, modelo, ficha completa, predicciones de C y prueba de invariancia superada	Falta la ficha o el entorno no es reproducible	predecir.py no corre o reentrena
Defensa	Cada integrante explica con solvencia una decisión ajena	Algún integrante solo explica su parte	No se presenta a la defensa

Durante la defensa oral cada integrante debe poder explicar cualquier parte del proyecto.

Penalizaciones

- Si se confirma una fuga de datos en las features, pierdes todos los puntos de 1.3 y las métricas afectadas no cuentan en 1.4 ni en 2.1.
- Si usas la etiqueta del corte B más de una vez, o ajustas algo después de verla, pierdes la mitad de 2.2.
- Si las predicciones del corte C cambian al agregar los datos posteriores, pierdes la mitad de 2.4.
- Una métrica reportada sin su línea base al lado no cuenta.

Calendario y entregas

Fecha	Qué pasa	Qué entregas
Miércoles 28 de octubre	Entrega de la Fase 1	Repositorio con la etiqueta fase-1 e informe de la Fase 1 (secciones 1.1 a 1.4)
Lunes 9 de noviembre	Entrega de la Fase 2	Repositorio con la etiqueta fase-2, informe de la Fase 2, predicciones del corte C y presentación